CS253 Assignment Report
Memory-Efficient Versioned File Indexer

Student Name: Ninad Tagade
Roll Number: 240699

1 Introduction / Overview

This project implements a memory-efficient versioned file indexer in C++. The program processes large text files using a fixed-size buffer and builds an index of word frequencies.

Instead of loading the entire file into memory, the program reads the file chunk by chunk. This approach allows the system to handle large log files efficiently while using limited memory.

The program supports the following queries:

- **Word Frequency Query** – Counts how many times a word appears in a file version.
- **Difference Query** – Computes the difference in frequency of a word between two file versions.
- **Top-K Query** – Displays the most frequent words in a file.

The implementation demonstrates the use of object-oriented programming concepts such as abstraction, polymorphism, templates, and exception handling.

2 Design and Approach

The program is designed using a modular object-oriented architecture in which each component of the system is responsible for a specific task. The main goal of the design is to efficiently process large text files while keeping memory usage low. This is achieved by combining buffered file reading, tokenization, indexing, and query processing into separate components that interact with each other in a well-defined pipeline.

The overall approach of the program can be divided into four main stages: file reading, tokenization, indexing, and query execution.

2.1 Buffered File Reading

The first step in the process is reading the input file. Instead of loading the entire file into memory, the program reads the file using a fixed-size buffer. The buffer size is specified by the user and is restricted to the range of 256 KB to 1024 KB. This restriction ensures that the program operates within reasonable memory limits.

The `BufferedFileReader` class handles all file input operations. It opens the file in binary mode and repeatedly reads chunks of data into a dynamically allocated buffer. After each read operation, the number of bytes actually read from the file is determined using the `gcount()` function. The contents of the buffer are then transferred to a string object so that they can be processed easily by the tokenizer.

This buffered reading approach allows the program to handle very large files efficiently.

2.2 Tokenization of File Content

Once a chunk of data has been read from the file, it is passed to the tokenizer. The `Tokenizer` class is responsible for identifying individual words in the text and preparing them for indexing.

Words are defined as sequences of alphanumeric characters. Any non-alphanumeric character acts as a delimiter that separates words. During tokenization, all characters are converted to lowercase so that the indexing process becomes case-insensitive.

A key challenge in buffered file processing is that a word may be split between two chunks. To address this issue, the tokenizer maintains a variable called `leftover`. If the end of a chunk contains a partial word, that portion is stored in the leftover variable and combined with the beginning of the next chunk before tokenization continues. This ensures that words spanning multiple chunks are processed correctly.

2.3 Building the Versioned Index

After tokenization, each extracted word is passed to the indexing component. The `VersionedIndex` class stores the frequency of each word using an `unordered_map`. The key of the map represents the word, while the value represents the number of times the word appears in the file.

The index is built incrementally as the file is processed chunk by chunk. Each time the tokenizer identifies a word, the corresponding count in the map is updated. Since the map allows constant average-time insertion and lookup, this approach provides efficient indexing even for large vocabularies.

Each input file is treated as a separate version, and therefore each version has its own independent index. This allows the program to compare different versions of a file when executing difference queries.

2.4 Query Processing

After the indexing stage is completed, the program executes the query specified by the user. The query system is designed using runtime polymorphism. An abstract base class called `QueryProcessor` defines a common interface for executing queries through a pure virtual function called `execute()`.

Different query types are implemented using derived classes. These include:

- **WordQueryProcessor** for retrieving the frequency of a specific word in a version.
- **DiffQueryProcessor** for computing the difference in frequency of a word between two versions.
- **TopKQueryProcessor** for displaying the most frequent words in a version.

At runtime, the program determines which query processor to create based on the command-line arguments provided by the user. The selected processor executes the required query using the previously constructed indices.

2.5 Execution Flow

The complete execution flow of the program is summarized as follows:

1. Parse command-line arguments and determine the query type.
2. Create a `VersionedIndex` object for each file version.
3. Read the file using `BufferedFileReader`.
4. Pass each chunk of data to the `Tokenizer`.
5. Update the index using extracted words.

6. Create an appropriate `QueryProcessor`.

7. Execute the query and display the results.

This modular design improves maintainability, readability, and extensibility of the code. New query types or processing features can be added easily without modifying the existing indexing mechanism.

3 Description of Classes

3.1 BufferedFileReader

The BufferedFileReader class is responsible for reading files using a fixed-size buffer.

Its main functions include:

- Opening the file in binary mode.
- Allocating memory for the buffer.
- Reading the file in chunks.
- Returning the number of bytes read in each iteration.

The buffer size is restricted between 256 KB and 1024 KB. The destructor ensures that the file is closed and the allocated memory is released properly.

3.2 Tokenizer

The Tokenizer class processes chunks of text and extracts valid words.

Key features include:

- Words are defined as sequences of alphanumeric characters.
- All characters are converted to lowercase to ensure case-insensitive indexing.
- A leftover variable is used to store partial words that may be split across buffer boundaries.

This mechanism ensures correct word extraction even when words span across two buffers.

3.3 VersionedIndex

The VersionedIndex class stores the word frequency index for a particular file version.

It uses an `unordered_map<string, int>` to maintain word frequencies efficiently.

The class provides methods to:

- Add words to the index.
- Retrieve the frequency of a specific word.
- Access all stored word frequencies.

Each file is treated as a separate version with its own index.

3.4 QueryProcessor

QueryProcessor is an abstract base class that defines a common interface for executing queries. It contains a pure virtual function `execute()` which must be implemented by derived classes.

This allows runtime polymorphism so that different query types can be handled using a common interface.

3.5 Derived Query Classes

Three derived classes implement specific query types.

WordQueryProcessor

This class calculates the frequency of a specified word in a given file version.

DiffQueryProcessor

This class compares two file versions and computes the difference in frequency of a given word.

TopKQueryProcessor

This class identifies the top K most frequent words in a version by sorting the stored word frequencies.

4 File Processing Using Fixed-Size Buffer

The program processes files using a buffered reading technique.

The steps involved are:

1. A buffer of size between 256 KB and 1024 KB is allocated.
2. The file is read into the buffer using `ifstream.read()`.
3. The number of bytes read is obtained using `gcount()`.
4. The data is converted into a string chunk.
5. The chunk is passed to the tokenizer for processing.

Since a word may be split between two chunks, the tokenizer temporarily stores the partial word in a leftover variable and continues processing it in the next chunk.

This approach allows the system to process very large files without loading them completely into memory.

5 Assumptions and Observations

The implementation makes the following assumptions:

- Words consist only of alphanumeric characters.
- Word matching is case-insensitive.
- Input files are plain text files.
- Buffer size must be between 256 KB and 1024 KB.

Observations from the implementation:

- Buffered reading significantly reduces memory usage.

- The use of `unordered_map` enables efficient word frequency updates.
- Sorting enables efficient identification of top-K words.
- The object-oriented design makes the system modular and extensible.